

CubeMars AK60-6 V3.0 KV80

Internal notes – MIT protocol, wiring, calibration,
servo-mode vs. MIT-mode, troubleshooting

Repo `exo-actuator-sensor-drivers`

May 21, 2026

Contents

1	What it is	2
2	What is the “MIT CAN protocol”?	2
2.1	Background	2
2.2	What is “T-Motor CAN”?	2
2.3	Required physical connection	3
2.4	MIT CAN frame structure	3
2.4.1	Command frame (host → motor)	3
2.4.2	Feedback frame (motor → host)	3
2.4.3	Special command frames	4
3	Two operating modes – the critical distinction	4
4	Bench bring-up and the CAN-ID discovery problem	5
4.1	Timeline	5
4.2	What the extended CAN ID means	5
4.3	Action required	5
5	Calibration procedure	5
5.1	Required steps (one-time after assembly or firmware flash)	6
5.2	Software zero position	6
6	MIT control modes	6
6.1	Recommended gains for AK60-6	6
6.2	Impedance / kinesthetic teaching	7
7	Multi-motor hardware wiring and power distribution	7
7.1	Single motor (bench configuration)	7
7.2	Two motors (parallel configuration)	7
7.2.1	Power distribution (star-point grounding)	7
7.2.2	Power supply sizing	8
7.2.3	CAN bus termination (two motors)	8
7.2.4	Configuration for two motors in Python	8
8	HDSC USB-to-CAN adapter – frame envelope	9
8.0.1	Transmit envelope (30 bytes, host → adapter)	9
8.0.2	Receive frame (16 bytes, adapter → host)	9

9	Python driver architecture	9
9.1	Context-manager idiom (all tests)	10
9.2	Test script inventory	10
10	Known issues and gotchas	10
11	TMotorCANControl reference library	11
11.1	How it differs from our driver	11
11.2	Using TMotorCANControl (reference, not our primary path)	11
12	V3.0 driver-board protocol (AK Series Manual V3.0.1)	11
12.1	Unified extended-ID layout	11
12.2	R-Link “CAN Mode” setting	12
12.3	Force-control (MIT) frame layout – V3.0 byte order	12
12.4	Parameter ranges (manual §4.2 table)	12
12.5	Feedback frame (extended ID 0x2968 for ESC=0x68)	13
12.6	Worked example – send MIT torque 1 Nm, ESC_ID = 104	13
12.7	Python driver: <code>src/ak_v3_can.py</code>	13
12.8	Compatibility note	13
13	Quick-start checklist	14

1 What it is

The **CubeMars AK80-9 KV60** is a brushless servo motor with an integrated planetary gearbox and dual magnetic encoders, designed for force-torque controlled robotic joints. It uses the same **MIT mini-cheetah CAN protocol** (hereafter *MIT mode*) that was published by the MIT Biomimetic Robotics Lab for their mini-cheetah actuators and subsequently adopted by many vendors (CubeMars, T-Motor, Damiao, and others).

Property	Value
Model	AK60-6 V3.0 KV80
Voltage	18–52 V (rated 48 V)
Rated current	10 A
Peak current	30 A
Gear ratio	6:1 planetary
Output torque (cont.)	≈ 5 N·m
Output torque (peak)	15 N·m
Output speed (no-load)	≈ 45 rad/s at 48 V
Position range	± 12.5 rad (firmware MIT limit, P MAX)
Encoder	dual 14-bit magnetic (rotor + output shaft)
CAN bus	1 Mbps standard or extended frame
CAN mode (our driver)	MIT mode only (see Section 3)
CAN ID (bench)	0x68 = 104 (confirmed via scan 12 May 2026)
R-Link MIT setting	Inquiry Feedback (not “Periodic Feedback”)
Configuration tool	R-Link (CubeMars Windows application)

2 What is the “MIT CAN protocol”?

2.1 Background

The MIT mini-cheetah actuator protocol was developed at MIT’s Biomimetic Robotics Lab and first described in:

Ben Katz, *A Low Cost Modular Actuator for Dynamic Robots*, M.Eng. thesis, MIT, 2019.

The protocol sends one 8-byte CAN 2.0A standard frame per motor command cycle. The frame packs five control parameters – position, velocity, stiffness (K_p), damping (K_d), and feed-forward torque – into a single tight bitfield. The motor responds immediately with an 8-byte feedback frame containing position, velocity, and torque.

This is **not** the same as CANopen, UAVCAN/DroneCAN, or any other higher-level protocol; it is a bare, vendor-specific bit-packing scheme over standard CAN 2.0A frames.

2.2 What is “T-Motor CAN”?

T-Motor CAN is the popular name for a close variant of the MIT mini-cheetah protocol used by **T-Motor** / **CubeMars** products. The name comes from the **T-Motor** brand (same parent company as CubeMars) and is also referred to as:

- *MIT mode* (by CubeMars R-Link and documentation)
- *Mini-Cheetah protocol* (by the robotics community)
- *TMotorCANControl API* (by the open-source **neurobionics/TMotorCANControl** library, which is included as a reference in this repo under **TMotorCANControl/**)

All of the following motors use the same protocol: T-Motor AK series, CubeMars AK series, Damiao DM series (with minor frame differences), and many other MIT-derivative actuators.

2.3 Required physical connection

Layer	Hardware	Notes
Host interface	HDSC USB-to-CAN adapter (USB-ID 2e88:4603)	Appears as <code>/dev/ttyACM0</code> on Linux
Host-to-adapter	USB 2.0 full-speed serial at 921600 bps	The adapter does USB-UART internally
Adapter-to-motor	CAN bus, two wires: CAN H and CAN L	Twisted pair or shielded cable preferred
Termination	$120\ \Omega$ resistor between CAN H and CAN L	Required at <i>both ends</i> of the bus
CAN bitrate	1 Mbps	Must match R-Link configuration
Power	18–52 V DC, ≥ 30 A capable supply	Separate from the logic/USB supply



Figure 1: Physical connection chain. The Python driver writes 30-byte HDSC envelope frames to the USB serial port; the adapter converts them to CAN 2.0A standard frames on the CAN bus.

2.4 MIT CAN frame structure

2.4.1 Command frame (host → motor)

One 8-byte payload packed into standard CAN frame with `CAN_ID = ESC_ID`:

Byte(s)	Contents	Bit layout
0	Position upper	<code>q[15:8]</code>
1	Position lower	<code>q[7:0]</code>
2	Velocity upper	<code>dq[11:4]</code>
3	Velocity lower + K_p upper	<code>dq[3:0] Kp[11:8]</code>
4	K_p lower	<code>Kp[7:0]</code>
5	K_d upper	<code>Kd[11:4]</code>
6	K_d lower + τ upper	<code>Kd[3:0] tau[11:8]</code>
7	τ lower	<code>tau[7:0]</code>

Bit-widths: position 16-bit, velocity 12-bit, K_p 12-bit, K_d 12-bit, τ 12-bit. All scaled linearly:

$$\text{uint} = \left\lfloor \frac{x + X_{\max}}{2 X_{\max}} \cdot (2^N - 1) \right\rfloor \quad \text{where } X_{\max} \in \{P_{\max}, V_{\max}, T_{\max}\}$$

For AK60-6: $P_{\max} = 12.5$ rad, $V_{\max} = 45$ rad/s, $T_{\max} = 15$ N·m.

2.4.2 Feedback frame (motor → host)

The motor replies with `CAN_ID = ESC_ID` (not a separate master address as with the Damiao motor):

Byte	Contents	Notes
0	Motor <code>ESC_ID</code>	Redundant with CAN ID
1–2	Position (16-bit)	same scaling as command
3–4	Velocity (upper 8 + lower 4)	12-bit, $\pm V_{\max}$
4–5	Torque (lower 4 + 8)	12-bit, $\pm T_{\max}$
6	MOSFET temperature (°C)	not decoded in current driver
7	Rotor temperature (°C)	not decoded in current driver

2.4.3 Special command frames

Eight-byte payloads with the last byte set to a magic value; the other seven bytes are 0xFF:

Payload (hex)	Function
FF FF FF FF FF FF FC	Enable motor (torque ON)
FF FF FF FF FF FF FD	Disable motor (torque OFF)
FF FF FF FF FF FF FE	Set current position as zero

3 Two operating modes – the critical distinction

The AK80-9 has **two fundamentally different CAN protocols** that cannot be mixed. The mode is stored in flash by R-Link and takes effect after a power cycle.

	MIT Mode	Servo Mode (“Periodic Feedback”)
R-Link setting	CAN Mode = Inquiry Feed-back	CAN Mode = Periodic Feed-back
CAN frame type	Standard 2.0A (11-bit ID)	Extended 2.0B (29-bit ID)
CAN ID scheme	Command: <code>ESC_ID</code> Feedback: <code>ESC_ID</code>	Feedback: $(\text{CMD_TYPE} \ll 8) \mid \text{ESC_ID}$
Frame format	8-byte MIT bit-packed	VESC/FESC floating-point extended protocol
Our driver	<code>cubemars_bus.py</code> (supported)	Not supported (extended protocol needed)
TMotorCANControl	<code>MIT_CAN</code> class (supported)	<code>servo_can</code> class
Feedback timing	On request (reply to MIT poll)	Autonomous at configured rate (50 Hz default)

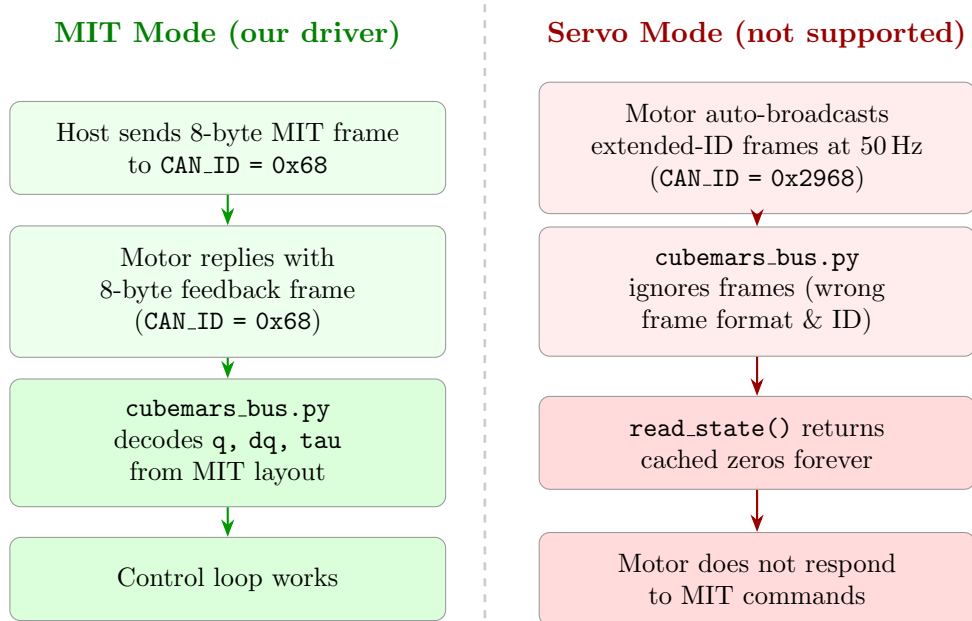


Figure 2: Divergent behaviour depending on the CAN Mode setting in R-Link. The bench motor was found in Servo Mode on 12 May 2026 during initial bring-up; see Section 4 for the diagnostic.

4 Bench bring-up and the CAN-ID discovery problem

4.1 Timeline

During initial bring-up (12 May 2026) the motor was found to be broadcasting extended-CAN frames at all times but not responding to MIT poll commands. Investigation steps:

1. **R-Link screenshot showed:** CAN Mode = Periodic Feedback, CAN ID = ID: 4. This placed the motor in *Servo mode*, not MIT mode.
2. **MIT poll scan** (`tests/cubemars/00_scan_can_id.py`) sent a null MIT frame ($K_p=K_d=\tau=0$) to every CAN ID in `0x01..0x7F`. Result: 1087 frames received, all from extended CAN ID `0x2968`.
3. Decoding the extended ID: $0x2968 = (0x29 \ll 8) \mid 0x68$ where `0x29` is the VESC/CubeMars servo status command type and `0x68 = 104` is the motor's ESC_ID.
4. **Conclusion:** motor ESC_ID is **104 (0x68)**. The CAN ID shown in R-Link (ID: 4) was from a previous *different* configuration page and was stale.

4.2 What the extended CAN ID means

In Servo mode, the motor uses the VESC/FESC extended-CAN protocol:

Field	Value
Full extended CAN ID (29-bit)	<code>0x2968 = 10600</code>
Command type byte (<code>cmd = id >> 8</code>)	<code>0x29 = 41 = COMM.GET_VALUES</code>
Motor ESC_ID byte (<code>mid = id & 0xFF</code>)	<code>0x68 = 104</code>
Broadcast rate	50 Hz (as configured in R-Link)

4.3 Action required

To use `cubemars_bus.py` (MIT mode), open R-Link and apply:

R-Link field	Current value	Required value
CAN Mode	Periodic Feedback	Inquiry Feedback (= MIT mode)
CAN Bitrate	1 Mbps	1 Mbps (keep)
CAN ID	4	104 (or any value – update <code>motor_config.py</code>)

Click **Write**, then **power-cycle the motor** (the mode change takes effect on reboot, not immediately).

After switching, rerun the scan:

```
1 python tests/cubemars/00_scan_can_id.py --start 1 --end 127
```

You should see one MIT feedback frame at the configured ESC_ID (not an extended CAN ID), and the decoder will report valid `q`, `dq`, `tau`.

5 Calibration procedure

Unlike the Damiao motor, the AK80-9 has **no firmware parameter read API** accessible over CAN. All calibration steps use R-Link.

5.1 Required steps (one-time after assembly or firmware flash)

1. **Motor Identification** – R-Link spins the motor in both directions to measure winding resistance, inductance, and back-EMF constant. Motor must be unloaded (disconnected from gearbox output if possible, or at least free to spin).
2. **Encoder Identification** – R-Link samples the encoder at many positions and builds a lookup table for pole-pair offset. Must be done immediately after Motor Identification.
3. **Write Parameters** – Click *Write* in R-Link to flash the identified values. Save `.AppParams` and `.McParams` files as backups.

Re-calibrate after:

- Swapping any two of the three motor phase wires.
- Replacing the motor or gearbox.
- Flashing new firmware.

5.2 Software zero position

`cubemars_bus.py` supports software-latching of the current angle as 0 rad via the special frame [FF FF FF FF FF FF FE]:

```
1 with open_cubemars_bus(set_zero=True) as bus:
2     # motor's current angle is now 0 rad for this session
3     bus.write("goal_position", {"j1": 1.0}, kp=30, kd=1.0)
```

This is volatile – it resets on power-cycle.

6 MIT control modes

The five MIT parameters give access to three practical control modes by choosing which are nonzero:

Mode	K_p	K_d	q_{des}	$\dot{q}_{\text{des}}$
Position hold	> 0	> 0	target rad	0
Velocity tracking	0	> 0	0	target rad/s
Open-loop torque	0	0	0	0

Warning: never command $K_p > 0$, $K_d = 0$ in position mode. Without damping the motor oscillates or runs away.

6.1 Recommended gains for AK60-6

Task	K_p	K_d	Notes
Stiff position hold	80–150	2.0–4.0	Heavy load; increase K_d first
Compliant hold	30–60	1.0–2.0	Lighter load or back-drivable
Impedance (teaching)	0–10	0.3–0.8	Very compliant; record demos
Velocity tracking	0	1.0–3.0	$K_p = 0$ mandatory
Torque (open-loop)	0	0	Max 1 N·m unloaded (T_MAX=15 N·m)

6.2 Impedance / kinesthetic teaching

To make the motor back-drivable for demonstration recording:

```
1 with open_cubemars_bus() as bus:
2     while recording:
3         q, dq, tau = bus.read_state()["j1"]
4         bus.write("goal_position", {"j1": q}, # spring at current pos
5                 kp=5.0, kd=0.5) # very soft
6         record(t, q, dq, tau)
```

Replay with higher stiffness:

```
1 bus.write("goal_position", {"j1": target_q}, kp=60, kd=2.0)
```

7 Multi-motor hardware wiring and power distribution

7.1 Single motor (bench configuration)

The bench configuration uses one AK60-6 motor connected to a Jetson Nano via a single HDSC USB-to-CAN adapter:

Signal / Power	Source	Destination
USB data	Jetson USB Type-A	HDSC USB Type-C
CAN H	HDSC adapter	Motor CAN connector
CAN L	HDSC adapter	Motor CAN connector
+48V	PSU (+) 25-30 A	Motor XT30 connector (+)
GND	PSU ()	Motor XT30 connector ()

Power supply requirements: 48V rated, $\geq 15\text{A}$ continuous. CAN termination: 120Ω resistor between CAN H and CAN L at the motor end.

7.2 Two motors (parallel configuration)

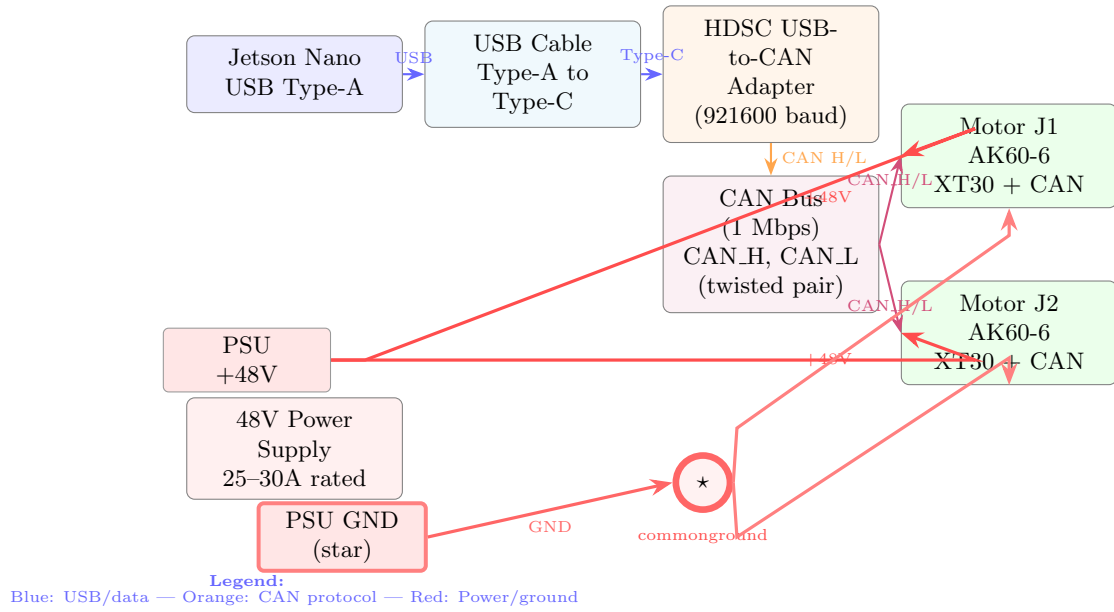
Two CubeMars motors can be operated simultaneously on a single CAN bus by assigning them different CAN IDs via R-Link. **Key principle:** both motors share the same CAN H/CAN L wires (daisy-chained) and receive power from the same 48V supply in parallel.

7.2.1 Power distribution (star-point grounding)

Signal	Topology	Notes
+48V	Parallel	Both motor XT30(+) connect to PSU(+)
GND (star)	Common junction	Both motor XT30() connect to single junction point ("star"), then to PSU()

The **star point** is a central ground junction where both motor return wires connect before returning to the PSU negative terminal. This topology prevents ground loops and reduces noise on the CAN bus (important for CAN signal integrity).

Complete wiring diagram (Jetson to dual motors with power distribution):



7.2.2 Power supply sizing

Motors	Rated current each	Total continuous
1×AK60-6	10 A	10 A
2×AK60-6	10 A	20 A

Recommendation: use a 48V power supply rated for *at least* 25–30 A continuous. This provides 1.25–1.5× safety margin above the 20 A sum and accommodates transient current spikes during acceleration.

7.2.3 CAN bus termination (two motors)

With two motors on the same CAN bus, place a single 120Ω termination resistor at the far end (typically at the last motor or at the HDSC adapter). Do *not* place terminators at both ends on a short bus (a few meters or less) as this can introduce reflection issues.

7.2.4 Configuration for two motors in Python

Update `src/motor_config.py` to define both motors:

```

1 DEFAULT_CUBEMARS_BENCH_CONFIG = CubeMarsBusConfig(
2     port="/dev/ttyACM0",
3     baudrate=921600,
4     motors={
5         "j1": CubeMarsMotorConfig(can_id=0x01, model="AK60-6"),
6         "j2": CubeMarsMotorConfig(can_id=0x02, model="AK60-6"),
7     },
8 )

```

Then in test scripts, use the same context-manager idiom:

```

1 with open_cubemars_bus() as bus:
2     # Read both motors in one pass
3     state = bus.read_state()
4     q1, dq1, tau1 = state["j1"]
5     q2, dq2, tau2 = state["j2"]
6
7     # Command both motors simultaneously
8     bus.write("goal_position",

```

```

9      {"j1": 1.57, "j2": -0.5},
10     kp=30, kd=1.0)

```

Each motor responds to its own CAN ID; the shared HDSC adapter multiplexes all frames on the same serial port.

Important: Run `tests/cubemars/00_scan_can_id.py` on each motor individually to confirm its CAN ID before adding it to the config dict.

8 HDSC USB-to-CAN adapter – frame envelope

The Python driver does not use a SocketCAN or PCAN interface; it writes a vendor-specific 30-byte serial envelope directly to the HDSC adapter. The adapter converts this internally to a CAN 2.0A standard frame.

8.0.1 Transmit envelope (30 bytes, host → adapter)

Bytes	Value (hex)	Meaning
0	55	Start-of-frame
1	AA	Start-of-frame
2	1E	Frame length = 30
3	03	Command type (send CAN frame)
4–8	01 00 00 00 0A	Fixed header fields
9–12	00 00 00 00	Reserved
13	CAN_ID[7:0]	CAN ID lower byte = ESC_ID
14	CAN_ID[10:8]	CAN ID upper bits (for IDs ≤0x7FF always 0)
15–20	00 00 08 00 00 00	Fixed (DLC=8)
21–28	payload	8-byte MIT payload
29	(computed)	Unused / padding

8.0.2 Receive frame (16 bytes, adapter → host)

Bytes	Value (hex)	Meaning
0	AA	Start byte
1	11	Command type (motor feedback)
2	08	DLC = 8
3–6	CAN ID (4-byte LE)	For MIT: CAN_ID = ESC_ID
7–14	payload	8-byte MIT feedback payload
15	55	End byte

The driver scans for 0xAA start / 0x55 end markers (16 bytes apart) and discards partial frames.

Servo-mode extended CAN ID. When the motor is in Servo Mode, the feedback CAN ID is 29-bit (0x2968 for our motor), but the HDSC adapter still delivers it in the 4-byte little-endian field (bytes 3–6). The driver compares against the configured ESC_ID (0x68) and these 4 bytes will never match (they decode to 0x00002968 = 10600), so *all servo-mode frames are silently discarded*. This is the root cause of the “all zeros” symptom.

9 Python driver architecture

File	Role
<code>src/motor_config.py</code>	Declarative config dataclasses + <code>DEFAULT_CUBEMARS_BENCH_CONFIG</code>
<code>src/cubemars.bus.py</code>	Self-contained MIT bus driver (<code>CubemarsMotorsBus</code>)
<code>src/_common.py</code>	<code>open_cubemars_bus()</code> context-manager helper
<code>tests/cubemars/</code>	Numbered test scripts 00–07

9.1 Context-manager idiom (all tests)

```
1 from _common import open_cubemars_bus
2
3 with open_cubemars_bus(set_zero=True) as bus:
4     bus.write("goal_position", {"j1": 1.57}, kp=30, kd=1.0)
5     q, dq, tau = bus.read_state()["j1"]
```

9.2 Test script inventory

Script	Purpose
00_loopback_dsd_tech.py	DSDTech USB adapter loopback (SW + physical)
00_loopback_socketcan.py	Jetson built-in CAN0 loopback (SW + physical)
00_scan_bitrate.py	Passive bitrate scan (find motor's actual rate)
00_probe_motor.py	Safe zero-torque CAN ping; confirms motor alive
00_sniff_motor.py	Passive V3.0 feedback sniffer; no TX (safest)
00_read_params.py	Read all motor parameters via UART serial (COMM_GET_VALUES)
01_check_connection.py	Poll 5 feedback frames, confirm non-zero
02_mit_position.py	MIT position hold, step through angles
03_mit_velocity.py	MIT velocity tracking
04_mit_torque.py	Open-loop torque (≤ 1 N·m unloaded)
05_kinesthetic_record.py	Hand-guide + save demo_trajectory_ak80.csv
06_replay_trajectory.py	Replay the recorded CSV
07_impedance_spring_return.py	Soft spring anchored at zero

Serial parameter read (00_read_params.py). Uses the motor's 3-pin UART port (not CAN) at 921 600 baud. Requires a USB-to-UART adapter (FTDI/CH340 etc.) wired: GND $\leftrightarrow$ GND, adapter TX $\rightarrow$ motor RX, adapter RX $\rightarrow$ motor TX. Sends COMM_GET_VALUES (0x45) and displays a table of all 17 operating parameters; also demonstrates the selective COMM_GET_VALUES_SETUP (0x10) bitmask query. Run as: `sudo python3 00_read_params.py [--port /dev/ttyUSB0]`.

10 Known issues and gotchas

1. **[CURRENT ISSUE] Motor is in Servo Mode.** The motor auto-broadcasts extended-CAN status frames at 50 Hz (CAN ID 0x2968, cmd 0x29, motor ID 0x68 = 104). `cubemars_bus.py` speaks MIT only (standard CAN). All feedback is silently discarded; `read_state()` returns zeros. **Fix:** R-Link $\rightarrow$ CAN Mode = **Inquiry Feedback** $\rightarrow$ Write $\rightarrow$ power-cycle. feedback frame is 104 (0x68). `motor_config.py` has been corrected to `can_id=0x68`.
2. **Model is AK60-6 V3.0 KV80, not AK80-9.** R-Link hardware/software version string AK60_6V / AK60_6_SE_V3 confirmed on 12 May 2026. All MIT limits updated to AK60-6 values: `CUBEMARS_LIMITS["AK60-6"] = [12.5, 45.0, 15.0]`.
3. **Never $K_d = 0$ in position mode.** Same rule as Damiao MIT mode: $K_d = 0$ with $K_p > 0$ removes all damping and causes oscillation or runaway.
4. **Calibration required before first use.** Run Motor Identification + Encoder Identification in R-Link with the motor unloaded. Write parameters. Re-run if any phase wire is swapped.
5. **No mode switch over CAN.** The MIT / Servo mode switch is a flash-stored setting only changeable via R-Link + power-cycle. It cannot be commanded over CAN at runtime.
6. **Peak current caution.** The motor allows up to 30 A peak. When testing open-loop torque (04_mit_torque.py), keep $\tau_{ff} \leq 1$ N·m when unloaded to avoid damaging a current-limited bench supply.

11 TMotorCANControl reference library

The `TMotorCANControl/` directory contains the neurobionics/`TMotorCANControl` open-source library, included as a vendored reference. It implements the same MIT CAN protocol for T-Motor / CubeMars actuators using **SocketCAN** (Linux `can0` interface via `python-can`).

11.1 How it differs from our driver

Aspect	<code>cubemars_bus.py</code> (ours)	<code>TMotorCANControl</code>
CAN interface	HDSC USB-to-CAN serial (<code>/dev/ttyACM0</code>)	SocketCAN (<code>can0</code> , <code>slcan</code> , etc.)
Dependency	<code>pyserial</code> only	<code>python-can</code> , SocketCAN kernel module
Jetson setup	Plug-in USB adapter, no kernel config	Requires <code>ip link set can0 up</code>
MIT protocol	Identical bit-packing	Identical bit-packing
Servo mode	Not supported	Supported via <code>servo.can</code> class
API style	LeRobot-style <code>read/write</code>	Iterator / context manager

11.2 Using TMotorCANControl (reference, not our primary path)

If you want to switch to SocketCAN on the Jetson (for lower latency or multi-motor daisy-chaining without the HDSC adapter):

```
# Bring up SocketCAN interface (HDSC adapter exposes slcan):
sudo slcand -o -c -s8 /dev/ttyACM0 can0
sudo ip link set can0 up type can bitrate 1000000

# Then use TMotorCANControl:
from TMotorCANControl.mit_can import TMotorManager_mit_can
with TMotorManager_mit_can(motor_type="AK80-9", device="can0",
                           motor_ID=104) as m:
    m.set_zero_position()
    m.update()
    print(m.position, m.velocity, m.torque)
```

Note: the HDSC adapter firmware does not natively speak `slcan`, so the above may require the `slcan` TTY line discipline. Our `cubemars_bus.py` avoids this by writing the HDSC proprietary 30-byte envelope directly, which is more reliable on Jetson without kernel module setup.

12 V3.0 driver-board protocol (AK Series Manual V3.0.1)

The CubeMars **V3.0** driver board uses a unified extended-frame CAN protocol that supersedes the older Damiao-style standard-frame MIT protocol assumed by `cubemars_bus.py` and by the `TMotorCANControl` library. Both servo modes and force-control (MIT) mode share one identifier scheme.

12.1 Unified extended-ID layout

$$\text{CAN_ID (29-bit)} = (\text{PACKET_TYPE} \ll 8) \mid \text{ESC_ID}$$

The packet type selects the control mode. All eight packet types share the same 29-bit ID slot and the same 8-byte DLC.

PACKET_TYPE	Name	Section	CAN_ID for ESC=0x68
0	SET_DUTY	§4.1.1	0x00000068
1	SET_CURRENT	§4.1.2	0x00000168
2	SET_CURRENT_BRAKE	§4.1.3	0x00000268
3	SET_RPM	§4.1.4	0x00000368
4	SET_POS	§4.1.5	0x00000468
5	SET_ORIGIN_HERE	§4.1.6	0x00000568
6	SET_POS_SPD	§4.1.7	0x00000668
8	SET_MIT	§4.2	0x00000868

12.2 R-Link “CAN Mode” setting

This selects feedback delivery only – *not* which control modes the firmware accepts. Both Servo and MIT commands work in either setting.

Inquiry Feedback	Motor replies with one feedback frame for each command received.
Periodic Feedback	Motor broadcasts feedback at CAN Fdb Rate (default 50 Hz).

12.3 Force-control (MIT) frame layout – V3.0 byte order

The V3.0 MIT byte order is **Kp first**, which differs from the classic mini-cheetah layout (position first) used by Damiao and earlier T-Motor firmware:

Byte	Content
0	Kp[11:4]
1	Kp[3:0] Kd[11:8]
2	Kd[7:0]
3	pos[15:8]
4	pos[7:0]
5	vel[11:4]
6	vel[3:0] tau[11:8]
7	tau[7:0]

Sub-modes (selected by which fields are nonzero).

- **Position:** $K_p > 0$, $K_d > 0$, set p_{des} . Never $K_d = 0$.
- **Velocity:** $K_p = 0$, $K_d > 0$, set v_{des} .
- **Torque :** $K_p = 0$, $K_d = 0$, set τ_{ff} .

12.4 Parameter ranges (manual §4.2 table)

Model	KV	$P_{\max}$ (rad)	$V_{\max}$ (rad/s)	$T_{\max}$ (N m)	K_p range	K_d range
AK10-9	60	12.56	28	54	0–500	0–5
AK60-6	80	12.56	60	12	0–500	0–5
AK70-9	60	12.56	30	32	0–500	0–5
AK80-9	100	12.56	65	18	0–500	0–5
AKE60-8	80	12.56	40	15	0–500	0–5
AKE80-8	30	12.56	20	35	0–500	0–5

For the bench AK60-6 V3.0 KV80: $V_{\max} = 60$ rad/s, $T_{\max} = 12$ N m. Note these differ from the values hard-coded in TMotorCANControl ($V_{\max} = 50$, $T_{\max} = 15$) and the older Damiao-style assumption ($V_{\max} = 45$, $T_{\max} = 15$).

12.5 Feedback frame (extended ID 0x2968 for ESC=0x68)

Bytes	Type	Meaning
0-1	int16_t	position ($\times 0.1$ deg)
2-3	int16_t	speed ($\times 10$ ERPM)
4-5	int16_t	current ($\times 0.01$ A)
6	int8_t	driver-board temperature ($^{\circ}\text{C}$)
7	uint8_t	error code (0 = no fault)

12.6 Worked example – send MIT torque 1 N m, ESC_ID = 104

Using the V3.0 layout, with $K_p = K_d = 0$, $p_{des} = v_{des} = 0$, $\tau_{ff} = 1$ N m for an AK60-6 ($T_{\max} = 12$):

$$\begin{aligned}t_{int} &= \text{float_to_uint}(1.0, -12, 12, 12) = 2218 \\p_{int} &= 32767 \quad (\text{centre of 16-bit unsigned, } p = 0) \\v_{int} &= 2047 \quad (\text{centre of 12-bit unsigned, } v = 0) \\kp_{int} &= 0, \quad kd_{int} = 0\end{aligned}$$

Resulting 8-byte payload: 00 00 00 7F FF 7F F8 AA, sent at extended CAN ID 0x00000868 (i.e. $(8 \ll 8) \mid 104$).

12.7 Python driver: src/ak_v3_can.py

A Python port of the official AK-V3.ino sketch is shipped at `src/ak_v3_can.py`. It uses `python-can` on top of the Linux SocketCAN interface (`can0`), which is exposed by the WaveShare / Canable USB-CAN adapter via the in-kernel `gs_usb` driver. Quick usage:

```
1 from ak_v3_can import AkV3Motor, ensure_can0_up
2 import can
3
4 ensure_can0_up(1_000_000)
5 bus = can.interface.Bus(channel="can0", bustype="socketcan")
6 motor = AkV3Motor(bus, can_id=104, model="AK60-6") # 104 = 0x68
7
8 # Position mode: hold 90 deg with moderate stiffness.
9 motor.set_mit(p_des=1.57, v_des=0.0, kp=30.0, kd=1.0, t_ff=0.0)
10
11 while True:
12     if motor.poll_feedback(timeout=0.05):
13         print(motor.pos_deg, motor.spd_erpm, motor.current_a)
```

12.8 Compatibility note

The bench-tested HDSC + `cubemars_bus.py` stack (standard 11-bit CAN ID, classic position-first MIT byte order) is *not* the protocol the V3.0 manual describes. Either:

- the bench firmware predates the V3.0.1 manual revision and still accepts the legacy MIT format on standard frames; or
- it accepts both formats in parallel.

For new code on a fresh V3.0 board, prefer `ak_v3_can.py`.

13 Quick-start checklist

1. Power on motor (18–52 V; bench uses 48 V).
2. Connect HDSC USB-to-CAN adapter to Jetson USB port.
3. Verify `ls /dev/ttyACM0` exists.
4. **Ensure motor is in MIT mode** (R-Link → CAN Mode = MIT, Write, power-cycle).
5. Run scan to confirm CAN ID:

```
conda activate actuators
cd tests/cubemars
python 00_scan_can_id.py
```

6. Expected: one MIT feedback frame at 0x68 (104) with valid `q`, `dq`, `tau`.
7. If scan shows extended-ID 0x2968: motor is still in Servo Mode. Return to step 4.
8. Run connection check:

```
python 01_check_connection.py
```

9. Proceed with higher test scripts (02–07).